

C/C++ Development Setup

Course Document

Setting Up Eclipse for C/C++ on Linux

This assumes you have already set up your Linux (Xubuntu) development environment.

Note: This guide assumes you have already installed Eclipse on your machine. See "Setting Up GitHub and Your Development Environments" in Course Information for how to set it up using the Xubuntu VM.

Note: These steps assume the Xubuntu VM. Eclipse installation can vary somewhat on other Linux distributions. If you are using a different distribution, you will need to adapt these steps yourself.

1. Install the C/C++ Toolchain

Recommendation: Windows does not include a C/C++ compiler, and there is no simple way to add one, unlike Linux or macOS. Getting GCC working natively on Windows means installing something like MSYS2 or WSL and manually configuring your PATH. I strongly encourage doing all C/C++ work in this course using Linux instead. If you choose to compile natively on Windows, you will need to figure out the setup yourself, and you will be on your own to troubleshoot any issues that come up.

1. Open a terminal and install the build tools and debugger:

```
sudo apt install build-essential gdb
```

Note: build-essential installs gcc, g++, make, and related tools. gdb is the debugger both Eclipse and VS Code use for breakpoints and stepping.

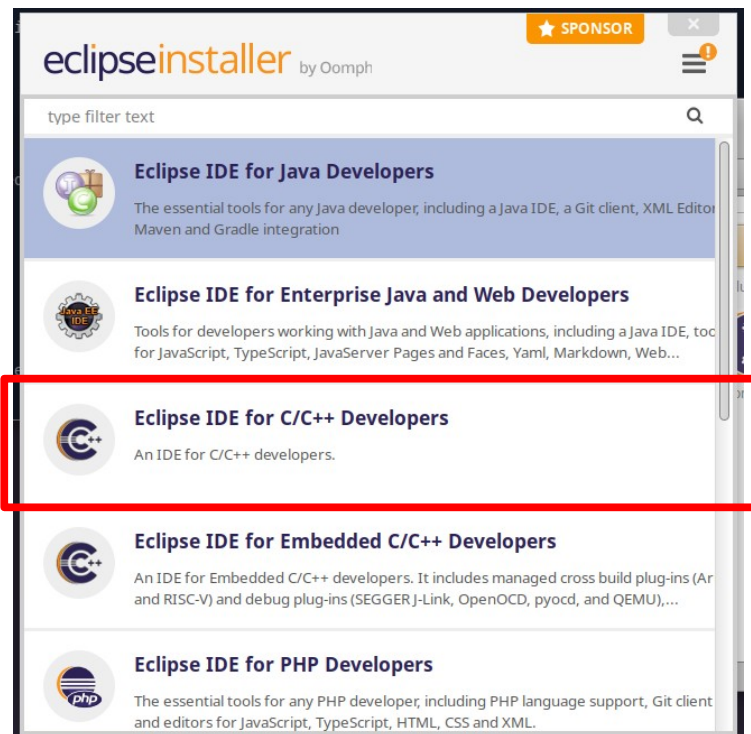
2. Set Up Eclipse for C/C++

Eclipse supports C/C++ development through a plugin called Eclipse CDT (C/C++ Development Tooling). If you already installed Eclipse for Java, choose one of the two options below.

Warning: Do not install Eclipse itself through your Linux package manager (apt). The packaged version tends to be older and does not include CDT. Use the official Eclipse Installer instead, the same one you used for the Java setup.

2a. Option A: Install a Separate C/C++ Eclipse

1. Run the Eclipse Installer again (the same one you used for Java).
2. Select "**Eclipse IDE for C/C++ Developers**" instead of the Java package.
3. Choose an installation folder separate from your Java Eclipse install, so the two do not conflict.
4. Click **Install**, accept the license, and click **Launch** when done.

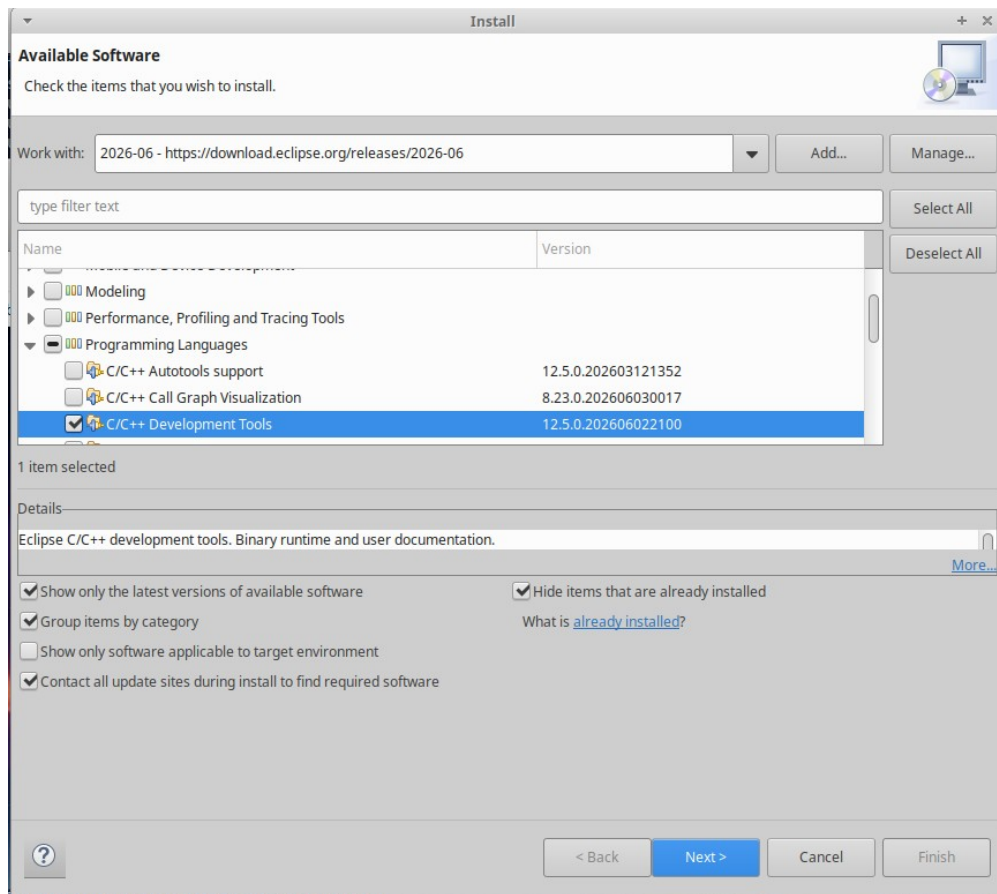


The Eclipse Installer package list; look for Eclipse IDE for C/C++ Developers

Note: Eclipse's own documentation recommends this approach: one Eclipse install per project type, so plugins from different languages do not mix.

2b. Option B: Add CDT to Your Existing Eclipse

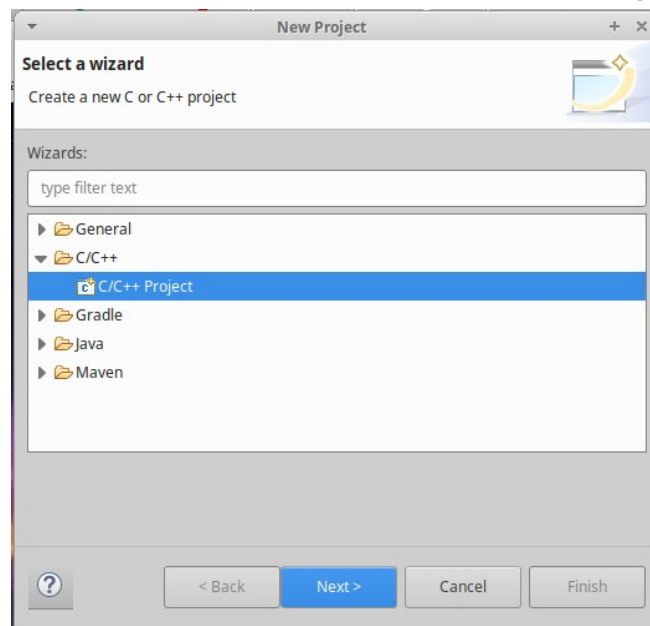
1. Open Eclipse (the same installation you used for Java).
2. Go to **Help > Install New Software**.
3. In the "Work with" dropdown, select the update site matching your installed Eclipse version.
4. Under **Programming Languages**, check **"C/C++ Development Tools."**
5. Click **Next**, accept the license, and click **Finish**.
6. Restart Eclipse when prompted.



C/C++ Development Tools checked in the Install New Software dialog

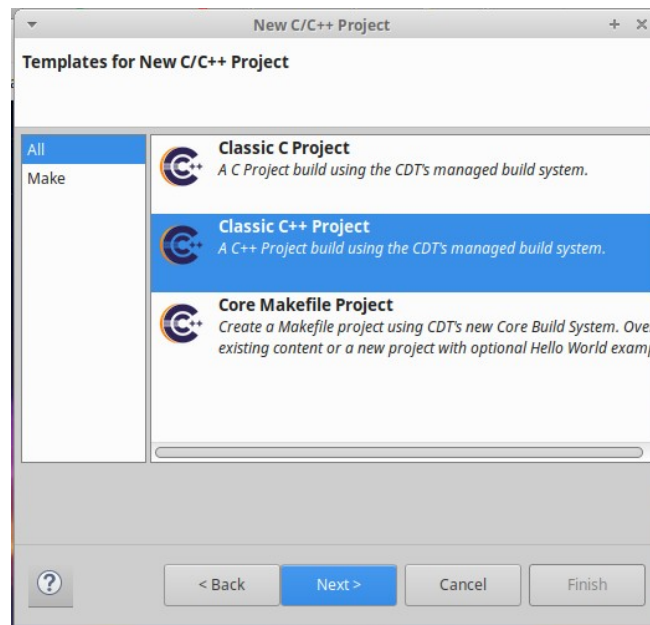
2c. Create and Test a C/C++ Project in Eclipse

1. Go to **File > New > Project**, then select **C/C++ > C/C++ Project**. Depending on your Eclipse version, this may instead appear directly as **File > New > C++ Project**.



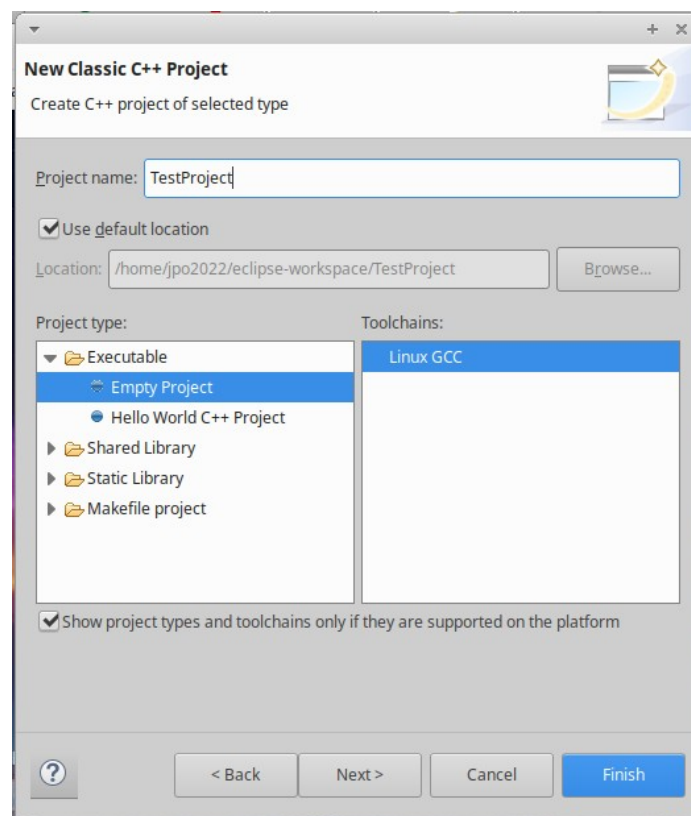
Selecting C/C++ Project in the New Project wizard

2. Select **Classic C++ Project**.



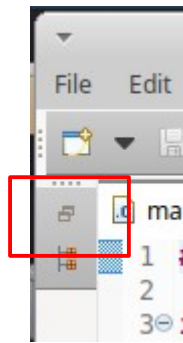
Selecting Classic C++ Project

3. Choose a project type, such as **Executable > Empty Project**, and select the **Linux GCC** toolchain.
4. Name the project.



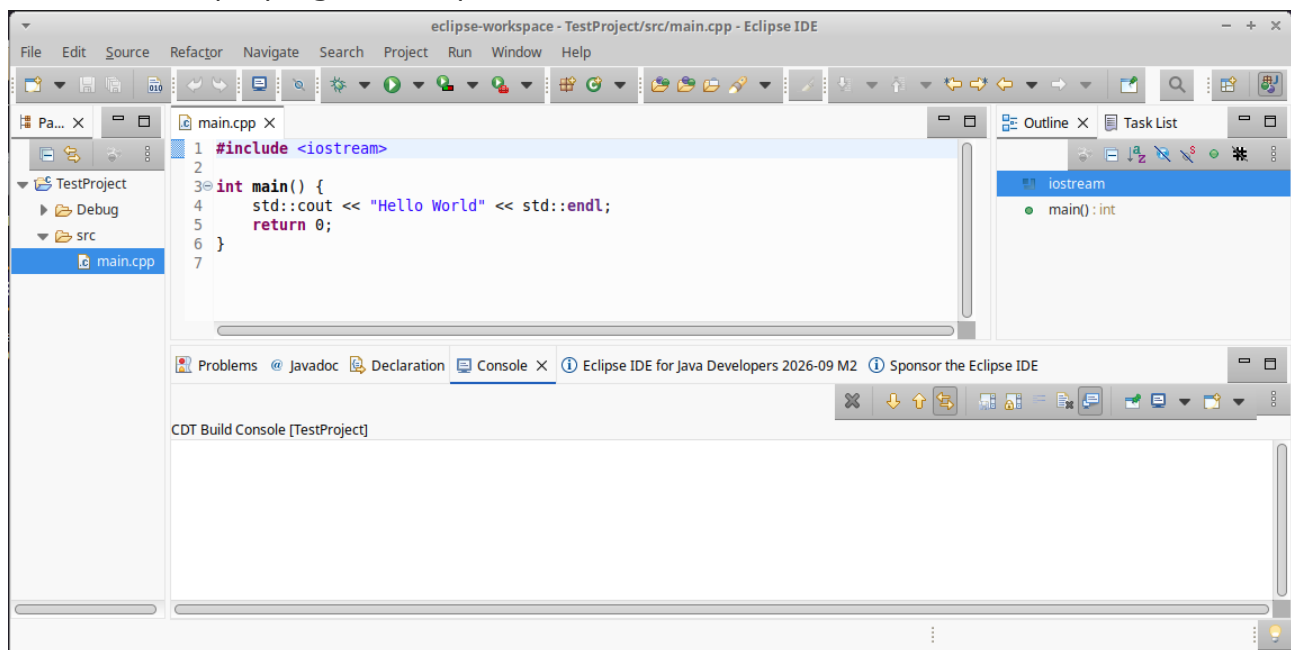
Naming the project and choosing Empty Project with the Linux GCC toolchain

5. If the Package Explorer pane is not visible, click the **Restore** button on the left side of the window to show it.



The Package Explorer pane restored

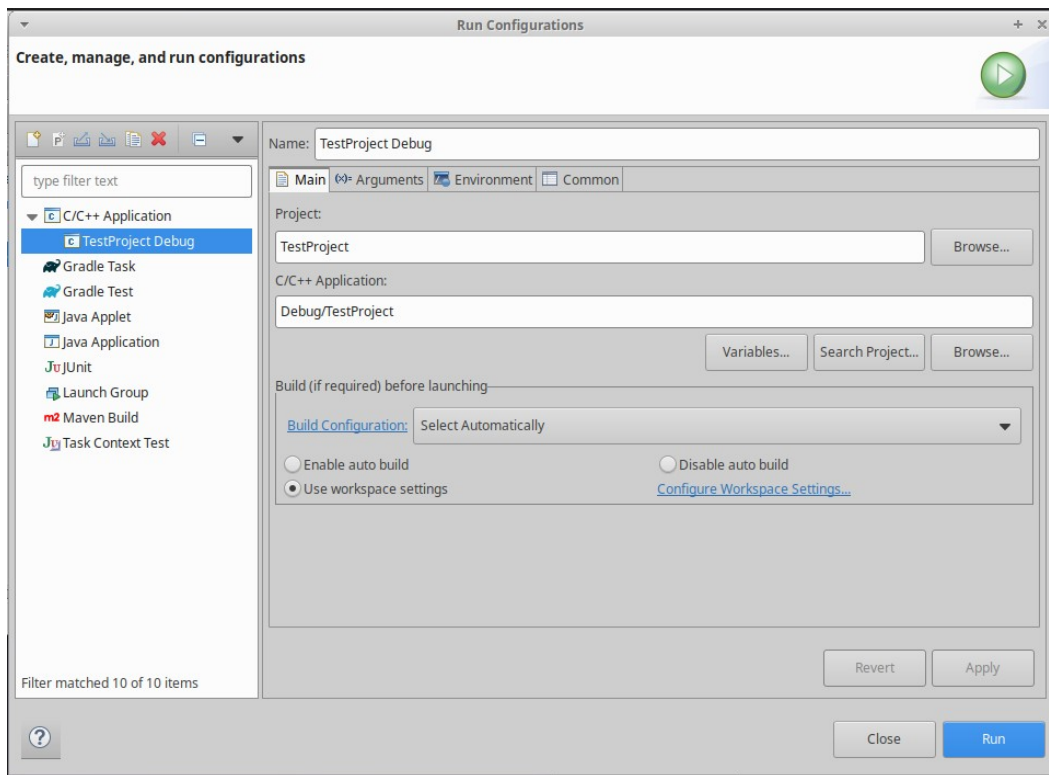
6. Create a folder named `src` in the project (right-click the project name > **New** > **Folder**).
7. Create a source file inside `src`, such as `main.cpp` (right-click `src` > **New** > **Source File**).
8. Write a simple program that prints "Hello World."



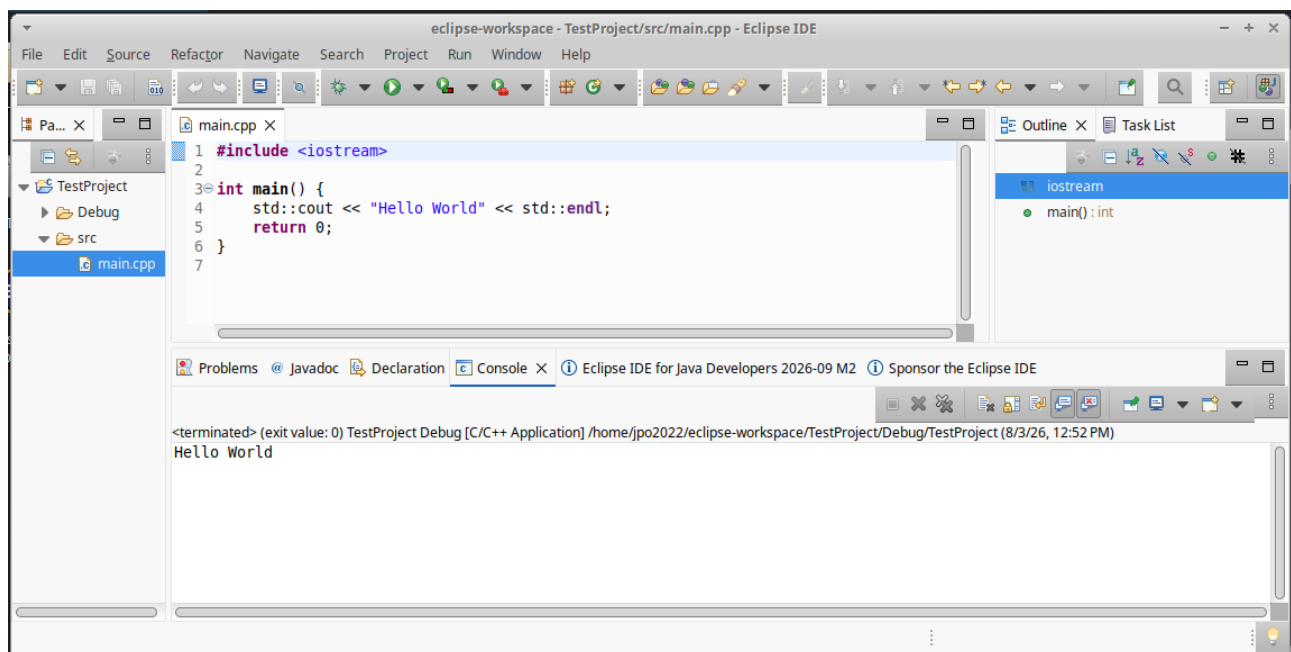
The Hello World program in main.cpp

9. Build the project (**Project** > **Build Project**, or the hammer icon).
10. Run it (**Run** > **Run As** > **Local C/C++ Application**).

Note: If no option appears under Run As, you will need to set up a run configuration manually instead. Go to **Run** > **Run Configurations**, right-click **C/C++ Application**, select **New Configuration**, choose your project, and click **Run**.



Setting up a run configuration manually



Hello World printed in the Console after running

3. Using a Different Editor or IDE

You may use VS Code, Vim, Emacs, or another editor or IDE instead of Eclipse if you prefer. Many working developers use a lightweight tool like Vim instead of a full IDE, and this class will use Vim during some sessions, so it is worth getting comfortable with it. I am unable to support VS Code this semester, but it remains an option if you want to use it. Whichever tool you use, your project must be organized the same way as the Eclipse project in Section 2 of this guide.